

SentinelGuard

Explication détaillée des mécanismes de sécurité utilisés dans le projet hackathon

Ce document complète l'explication d'architecture. Il explique concrètement ce que fait l'application côté sécurité : comment elle analyse les IP, comment elle détecte les connexions suspectes, comment elle journalise les événements, et ce que la détection de capture d'écran peut réellement faire.

Important : SentinelGuard est un MVP de démonstration. Il montre une logique de sécurité réaliste, mais ce n'est pas encore un produit de cybersécurité de niveau entreprise.

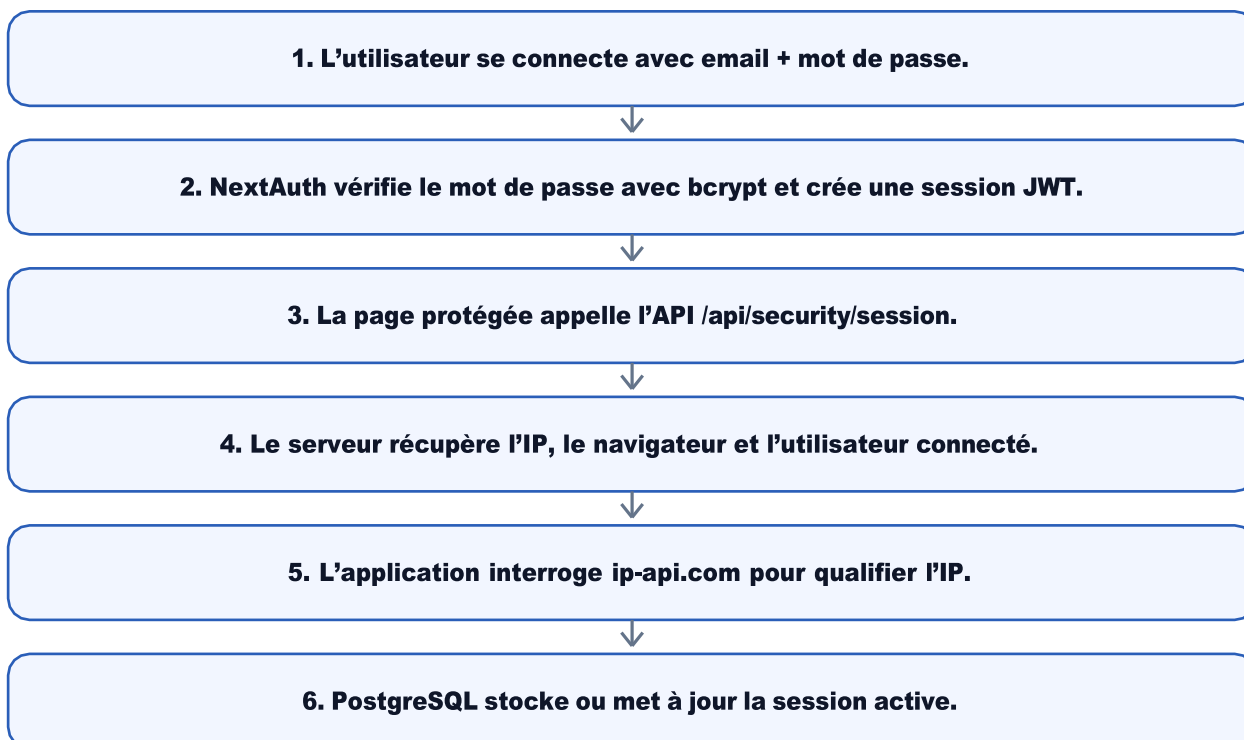
1. Résumé très simple

SentinelGuard simule une plateforme anti-fraude qui protège une page contenant des données sensibles. Lorsqu'un utilisateur se connecte, l'application vérifie plusieurs signaux : son IP, son appareil, les autres connexions actives, les IP bloquées et certains comportements comme une tentative d'impression ou de capture d'écran.

Élément vérifié	Comment c'est testé concrètement	Résultat attendu
Adresse IP	Récupération de l'IP dans les en-têtes HTTP puis appel à ip-api.com pour savoir si l'IP ressemble à un VPN, proxy ou datacenter.	Alerte VPN / Proxy si l'IP est suspecte.
IP bloquée	Comparaison de l'IP avec la table BlockedIP en base PostgreSQL.	Accès marqué comme critique si l'IP est présente dans la liste noire.
Connexions simultanées	Recherche des sessions actives du même utilisateur avec plusieurs IP différentes.	Alerte si le même compte est actif depuis plusieurs IP.
Capture / impression	JavaScript côté navigateur écoute certains raccourcis clavier et l'événement beforeprint.	Journalisation d'une tentative de capture ou d'impression.
Dashboard admin	Lecture des logs et statistiques depuis PostgreSQL.	Affichage des alertes, sessions, IP bloquées et graphiques.

En clair : le projet ne “devine” pas magiquement qu'une personne fraude. Il additionne plusieurs signaux techniques et crée des alertes quand un comportement ressemble à un risque.

2. Chaîne complète quand un utilisateur arrive sur la page protégée





7. Le système vérifie les IP bloquées et les connexions simultanées.



8. En cas d'anomalie, un événement est écrit dans SecurityLog.



9. Le dashboard admin affiche les alertes et statistiques.

Ce fonctionnement est déclenché automatiquement dès que l'utilisateur arrive sur la page protégée.

3. Sécurité de connexion : NextAuth + bcrypt + rôles

La connexion utilise NextAuth avec un provider "credentials", donc une authentification classique par email et mot de passe. Les mots de passe ne sont pas censés être stockés en clair : ils sont comparés avec bcrypt.

Composant	Rôle dans la sécurité
NextAuth	Gère la session utilisateur et protège les routes connectées.
CredentialsProvider	Permet la connexion avec email + mot de passe.
bcryptjs	Compare le mot de passe saisi avec le hash stocké en base.
JWT	Stocke l'état de session côté application sans table de session NextAuth dédiée.
role = admin / user	Sépare les droits : dashboard admin réservé aux administrateurs.
NEXTAUTH_SECRET	Sert à signer/chiffrer la logique de session. Il doit rester secret.

Exemple simple : un utilisateur classique peut accéder à l'espace protégé. Un administrateur peut en plus ouvrir le dashboard de supervision.

Limite actuelle : le projet n'intègre pas encore de MFA, de politique de mot de passe forte, de verrouillage après trop d'échecs, ni de rate limiting. Pour une vraie production, ce serait à ajouter.

4. Analyse des adresses IP

Le projet récupère l'adresse IP du visiteur à partir des en-têtes HTTP. Il lit d'abord x-forwarded-for, puis x-real-ip. Si l'IP est locale ou privée, par exemple 127.0.0.1, 192.168.x.x ou 10.x.x.x, elle n'est pas directement analysable comme une IP publique.

Ensuite, l'application appelle ip-api.com avec plusieurs champs : pays, ville, fournisseur Internet, organisation, mobile, proxy, hosting et IP analysée.

```
fields = status,message,country,countryCode,regionName,city,isp,org,as,mobile,proxy,hosting,query
```

Champ renvoyé	Utilisation dans le projet
country / city	Affichage de la localisation approximative.
isp / org	Affichage du fournisseur ou de l'organisation liée à l'IP.
proxy	Si true, le projet considère que l'IP peut être un VPN ou proxy.
hosting	Si true, l'IP est liée à un hébergeur/datacenter, souvent utilisé par des VPN ou bots.
mobile	Indique si l'IP est liée à un réseau mobile.
query	IP publique réellement analysée.

La règle principale est simple : si proxy = true ou hosting = true, alors l'IP est considérée comme suspecte.

5. Détection VPN / proxy : ce qui est réellement fait

Quand l'utilisateur arrive sur la page protégée, le backend appelle la fonction `checkIpReputation()`. Cette fonction interroge `ip-api.com`. Si l'API indique que l'IP est un proxy ou une IP d'hébergement, le projet crée une alerte `VPN_PROXY`.

```
if (reputation.isSuspicious) {
  type: 'VPN_PROXY'
  severity: 'high'
  message: 'VPN/Proxy détecté...'
}
```

Cas détecté	Niveau	Effet
IP marquée proxy	Élevé	Écriture d'un log <code>VPN_PROXY</code> .
IP de datacenter / hébergeur	Élevé	Écriture d'un log <code>VPN_PROXY</code> avec raison "hosting".
IP normale	Faible	Journalisation d'une connexion normale.

Attention : ce n'est pas une vraie base de threat intelligence comme AbuseIPDB, VirusTotal, MaxMind minFraud ou IPQualityScore. C'est une détection simple, suffisante pour une démo hackathon.

6. Blocage d'IP

Le projet possède une table PostgreSQL appelée `BlockedIP`. Un administrateur peut ajouter une IP à cette liste depuis le dashboard. À chaque connexion, l'application compare l'IP de l'utilisateur avec cette table.

```
const blocked = await prisma.blockedIP.findUnique({
  where: { ipAddress: reputation.ip }
})
```

Si l'IP est trouvée dans `BlockedIP`, le serveur renvoie une réponse bloquée et enregistre un événement `BLOCKED_IP` avec une sévérité critique.

Action admin	Stockage	Conséquence
Bloquer une IP	Création ou mise à jour dans <code>BlockedIP</code> .	Les futures connexions depuis cette IP sont signalées comme bloquées.
Débloquer une IP	Suppression de l'entrée dans <code>BlockedIP</code> .	L'IP n'est plus considérée bloquée.
Tentative depuis IP bloquée	Log <code>SecurityLog</code> de type <code>BLOCKED_IP</code> .	Alerte critique dans le dashboard.

7. Connexions simultanées

Le projet crée une session active dans la table `UserSession` à chaque passage sur la page protégée. Cette session contient l'utilisateur, l'IP, la localisation, l'appareil, le User-Agent, la date de création et la date `lastSeen`.

Ensuite, le serveur regarde toutes les sessions actives du même utilisateur. S'il trouve plusieurs IP différentes, il considère que le compte peut être partagé ou compromis.

```
const activeSessions = await prisma.userSession.findMany({
  where: { userId: user.id, isActive: true }
})
const distinctIps = Array.from(new Set(activeSessions.map(s => s.ipAddress)))
if (distinctIps.length > 1) { ... }
```

Exemple : Thomas est connecté depuis Lyon avec une IP Orange, puis le même compte apparaît en même temps depuis Amsterdam avec une IP datacenter. Le système crée une alerte `SIMULTANEOUS_LOGIN`.

Limite actuelle : il faut bien fermer ou expirer les sessions. Sinon une ancienne session peut rester active trop longtemps. Pour une vraie production, il faudrait ajouter une expiration automatique, par exemple inactive après 10 ou 15 minutes.

8. Détection de capture d'écran et impression

Un site web ne peut pas vraiment empêcher quelqu'un de prendre une capture d'écran avec son système d'exploitation, un téléphone, un outil externe ou certaines extensions. Le navigateur donne peu d'accès à ce niveau pour des raisons de sécurité et de vie privée.

Dans ce projet, la détection est donc une détection par signaux côté navigateur. Le composant React écoute certains événements clavier et l'événement d'impression.

Signal surveillé	Méthode utilisée	Ce que le projet fait
Touche Impr. écran	Événements keydown et keyup, e.key = PrintScreen.	Compteur + toast + log SCREENSHOT_ATTEMPT.
Raccourci macOS	Cmd + Shift + 3 / 4 / 5.	Log d'une tentative de capture.
Raccourci proche Windows	Shift + Ctrl/Cmd + S.	Log d'une tentative de capture, mais Win+Shift+S n'est pas garanti.
Impression page	Événement beforeprint.	Log d'une tentative d'impression/exfiltration.

```
window.addEventListener('keyup', onKey)
window.addEventListener('keydown', onKey)
window.addEventListener('beforeprint', onBeforePrint)

await fetch('/api/security/screenshot', { method: 'POST', ... })
```

Ce qui est stocké : le type d'événement, la méthode détectée, l'utilisateur, l'IP, la localisation et la date. L'application ne récupère pas l'image de la capture d'écran.

Formulation correcte pour la présentation : “Le projet ne bloque pas physiquement toutes les captures. Il détecte et journalise des tentatives probables de capture ou d'impression afin d'alerter l'administrateur.”

Ce qu'il ne faut pas dire : “On empêche totalement les captures d'écran.” Ce serait faux.

9. Journalisation des événements de sécurité

Tous les événements importants sont enregistrés dans la table SecurityLog. C'est le cœur du dashboard : sans logs, il n'y aurait rien à analyser.

Type de log	Quand il est créé	Sévérité
LOGIN	Connexion normale sans menace détectée.	low
VPN_PROXY	IP classée proxy ou hébergeur/datacenter.	high
SIMULTANEOUS_LOGIN	Même compte actif depuis plusieurs IP.	high
SCREENSHOT_ATTEMPT	Raccourci de capture ou impression détecté.	medium
BLOCKED_IP	IP bloquée ou tentative depuis IP bloquée.	medium à critical
INFO	Événement informatif, par exemple déblocage d'une IP.	low

Chaque log peut contenir des métadonnées JSON, par exemple proxy=true, hosting=true, fournisseur Internet ou liste des IP utilisées.

10. Dashboard admin

Le dashboard ne fait pas directement la détection. Il lit ce qui est déjà présent en base et le rend lisible pour un humain : statistiques, logs récents, sessions actives, IP bloquées et simulateur.

Écran	Données lues
Cartes statistiques	Total menaces, sessions actives, IP bloquées, alertes critiques.
Graphiques	Répartition par type de menace et activité des 7 derniers jours.
Logs	Événements SecurityLog triés du plus récent au plus ancien.
Sessions	Sessions actives avec IP, localisation, appareil et indicateur suspect.
IP bloquées	Liste BlockedIP, ajout et suppression.
Simulateur	Création manuelle de scénarios de menace pour la démo.

L'accès au dashboard est protégé par requireAdmin(). Si l'utilisateur n'est pas connecté ou n'a pas le rôle admin, l'API renvoie 401 ou 403.

11. Simulateur de menaces

Pour un hackathon, c'est très utile : on n'a pas besoin de vrais attaquants ou de plusieurs pays. Le simulateur crée de faux événements réalistes dans la base.

Scénario	Ce qu'il crée
Connexion simultanée	Deux sessions actives pour demo.utilisateur@sentinel.fr depuis deux IP/pays différents.
VPN / proxy	Une session active depuis une IP de démonstration et un log VPN_PROXY.
Capture d'écran	Un log SCREENSHOT_ATTEMPT avec méthode PrintScreen.
IP bloquée	Une IP est ajoutée à BlockedIP puis un log BLOCKED_IP est créé.

Le simulateur est réservé aux administrateurs. Il sert uniquement à montrer le fonctionnement pendant la présentation.

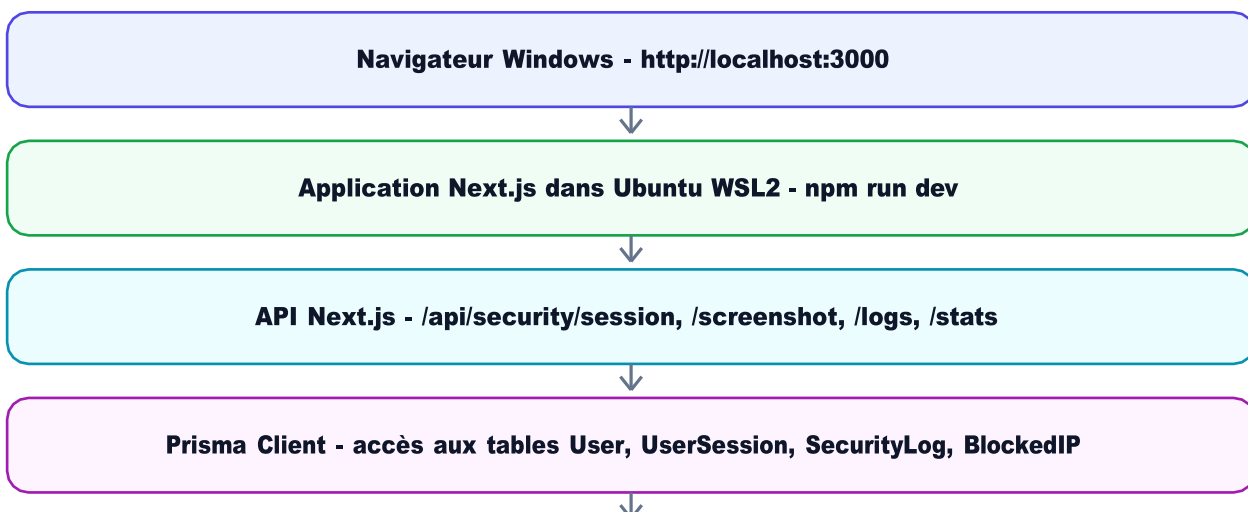
12. Stockage : ce qui est dans PostgreSQL

Table	Contenu	Utilité sécurité
User	Comptes, emails, hash de mot de passe, rôle.	Authentification et séparation admin/user.
UserSession	IP, localisation, appareil, userAgent, isActive, isFlagged.	Détection des connexions simultanées et suivi de session.
SecurityLog	Type, sévérité, message, IP, utilisateur, métadonnées, date.	Historique des alertes et alimentation du dashboard.
BlockedIP	IP bloquée, raison, origine du blocage.	Blocage ou signalement des IP interdites.

Prisma sert d'intermédiaire entre le code TypeScript/Next.js et PostgreSQL. Au lieu d'écrire toutes les requêtes SQL à la main, le code utilise `prisma.userSession.create()`, `prisma.securityLog.findMany()`, etc.

13. Hébergement local : qui fait quoi ?

Le projet tourne dans Ubuntu WSL2. WSL2 est un vrai environnement Linux intégré dans Windows. Docker Desktop fournit le moteur Docker et l'intègre à Ubuntu. La base PostgreSQL tourne dans un conteneur Docker.



PostgreSQL 16 dans Docker - port 5432 - volume postgres_data

14. Améliorations possibles après le hackathon

- Ajouter du MFA pour les comptes sensibles.
- Ajouter du rate limiting sur login et API.
- Ajouter une expiration automatique des sessions inactives.
- Remplacer ip-api.com par une vraie solution de threat intelligence.
- Ajouter AbuseIPDB, VirusTotal, IPQualityScore ou MaxMind selon le budget.
- Ajouter des règles de scoring : IP suspecte + nouvel appareil + pays inhabituel = risque élevé.
- Ajouter des webhooks ou notifications email/Discord/Slack pour les alertes critiques.

Conclusion

SentinelGuard est un bon projet hackathon parce qu'il est démontrable et compréhensible. Le cœur du projet n'est pas de faire une sécurité parfaite, mais de montrer une chaîne de détection complète : collecter des signaux, les analyser, stocker des preuves, puis permettre à un administrateur de réagir.